AURORA UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Backend Development Fundamentals — Standard Lab Report

Student Name:	A.VENKATA SAI	**Roll / Reg No:**	252U1R5010
Course / Code:	Backend Development Fundamentals	**Section / Room:**	AIDE 2A
Experiment No:	Lab Report 03	**Date of Execution:**	08/11/26
Faculty In-Charge:	Joel Prashanth D	**Academic Term:**	Year II, Term II

LAB 03: REST API, QUERY PARAMETERS, VALIDATION AND DEBUGGING

1. Problem Statement

Description:

Modern web and enterprise applications rely on Representational State Transfer (REST) APIs to decouple client applications from backend persistence engines. In Backend Development Fundamentals Lab 2, a foundational Node.js server with SQLite integration was created. However, practical backends require expressive data filtering capabilities, robust input validation to prevent invalid state propagation, precise HTTP status codes to guide client consumers, security safeguards against SQL injection, and systematic debugging practices to resolve runtime operational errors.

This laboratory assignment mandates converting the foundational Lab 2 backend project into an industry-standard, fully functional REST API. The system manages a book catalog resource (/books) through semantic HTTP methods (GET, POST, PUT, DELETE), supports dynamic query parameter filtering (search by title using SQL LIKE, allowlist-governed sorting, pagination with limit and offset, and bounded price range filtering), enforces rigorous payload schema validation for resource creation, adheres to consistent JSON response formatting, and systematically analyzes and resolves four critical backend failure modes.

Objective:

• Transform the SQLite and Express backend into an organized, modular RESTful architecture (routes, controllers, middlewares, config).
• Implement semantic, resource-based URL routing following the REST design principles.
• Implement safe and dynamic query parameter processing for substring searching (SQL LIKE) and price range filtering (BETWEEN).
• Enforce compile-time allowlists for sorting columns and sorting directions to prevent SQL injection vulnerabilities.
• Construct robust request body validation rejecting missing fields and invalid data types with HTTP 400 Bad Request.
• Standardize API responses using accurate HTTP status codes (200, 201, 400, 404, 500) and consistent JSON envelopes.
• Reproduce, analyze, and document verified solutions for four quintessential backend bugs (EADDRINUSE, wrong import path, SQL table typo, missing await).

• Conduct exhaustive verification across 18 Postman test scenarios with complete request/response evidence.

Requirements:

1. REST API Endpoint Naming: Resource-based routing (/books, /books/:id) paired with correct HTTP methods (GET, POST, PUT, DELETE).
2. Query Parameters: GET /books supporting search_q, order_by (with allowlist), order (ASC/DESC), limit, and offset with sensible defaults.
3. Price Range Filtering: min_price and max_price with SQL BETWEEN and unilateral boundary handling.
4. POST Request Validation: Strict validation requiring title, author, and numeric non-negative price.
5. HTTP Status Codes & Error Formats: Correct status assignment and standardized JSON structures.
6. Debugging Demonstrations: Empirical reproduction and resolution for EADDRINUSE, MODULE_NOT_FOUND, SQL table typo, and missing await.
7. Testing & Verification: Complete 18-case Postman test checklist and screenshot evidence.

Technologies Used:

• Runtime Environment: Node.js (v24.14.0, JavaScript ES6+ / CommonJS)
• Web Framework: Express.js (v4.x) with express.json() body parsing
• Persistence Layer: SQLite3 embedded relational database with sqlite promise wrapper
• Query Paradigm: Parameterized SQL queries with dynamic predicate construction
• Testing Tools: Postman Desktop Client v10+, Node.js Native Fetch Automated Test Suite
• Documentation & Formatting: Microsoft Word, Python docx/Pillow, Markdown

Input and Output Examples:

Operation / Action	HTTP Request & Input	HTTP Status	Output JSON Response Envelope
Filtered Book Retrieval	`GET /books?search_q=JavaScript&min_price=20`	200 OK	`{ "message": "Books retrieved successfully", "total": 2, "data": [{ "id": 3, "title": "JavaScript: The Good Parts", ... }] }`
Resource Creation	`POST /books { "title": "Mastering Full-Stack", "author": "Joel Prashanth D", "price": 49.99 }`	201 Created	`{ "message": "Book created successfully", "data": { "id": 11, "title": "Mastering Full-Stack", ... } }`
Validation Failure	`POST /books { "author": "Joel Prashanth D", "price": "free" }`	400 Bad Request	`{ "message": "Validation failed: The title field is required...", "errors": ["The title field is required..."] }`
Resource Not Found	`GET /books/99999`	404 Not Found	{

			"message": "Book not found" }

2. Implementation Steps

Step 1 (Project Architecture & Environment Initialization)

Organized the project workspace into a modular, clean MVC-inspired backend architecture. Established separate directories: config/ for database connectors and initialization; routes/ for HTTP verb and path definitions; controllers/ for business logic and SQL execution; middleware/ for request validation and centralized error interception; and data/ for SQLite database persistence. Initialized package.json and installed runtime dependencies: express, sqlite3, sqlite (async promise wrapper), and dotenv. Configured .gitignore to prevent committing node_modules, .env, and local database binaries.

Step 2 (Database Configuration & Schema Migration)

Designed config/database.js to establish a persistent SQLite connection with PRAGMA foreign_keys = ON;. Created a robust initialization routine that checks for the existence of the books table (columns: id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT NOT NULL, author TEXT NOT NULL, price REAL NOT NULL, created_at DATETIME DEFAULT CURRENT_TIMESTAMP). Implemented auto-seeding logic: if the table is freshly created and empty, it automatically seeds 10 diverse sample technical books with varied prices ($19.99 to $65.00) to support immediate filtering and sorting testing.

Step 3 (REST API Endpoint Naming & Routing)

Architected resource-oriented endpoint paths in routes/books.js. Replaced RPC-style naming (/getBooks, /addBook) with REST standards: GET /books (retrieve all/filtered), GET /books/:id (retrieve one), POST /books (create), PUT /books/:id (update), and DELETE /books/:id (delete). Mapped each route to its corresponding controller action, attaching validation middlewares before controller execution.

Step 4 (Query Parameter Implementation & Parameterized SQL LIKE)

Implemented dynamic query processing in controllers/booksController.js. For the search_q parameter, constructed a SQL LIKE clause: title LIKE ? bound with `%${search_q}%`. This enables case-insensitive substring matching without string concatenation. Implemented sorting via order_by and order. Because SQL drivers do not permit parameter binding on column identifiers and keywords, created a strict middleware allowlist validating order_by against ['id', 'title', 'author', 'price', 'created_at'] and order against ['ASC', 'DESC']. Added limit and offset with sensible defaults (10 and 0).

Step 5 (Price Range Filtering with SQL BETWEEN)

Implemented flexible price filtering supporting min_price, max_price, or both. If both boundaries are supplied, constructed the parameterized condition price BETWEEN ? AND ?; if only min_price is present, applied price >= ?; if only max_price is present, applied price <= ?. Enforced cross-parameter validation in middleware ensuring min_price is a non-negative number and min_price <= max_price, rejecting invalid ranges with HTTP 400 Bad Request.

Step 6 (Request Validation Middleware)

Engineered validateBook middleware for POST requests. Validated that title and author are non-empty strings after trimming, and price is a valid non-negative numeric value. Explicitly rejected non-numeric types (e.g. 'free', boolean flags, negative values) and missing keys. Upon detection of any invalid input, intercepted the request immediately and returned HTTP 400 Bad Request with a structured error array detailing all violated constraints.

Step 7 (HTTP Status Codes & Consistent JSON Formatting)

Standardized API responses across the entire system. Assigned HTTP 200 for successful GET, PUT, and DELETE operations; HTTP 201 for resource creation (POST); HTTP 400 for client validation failures; HTTP 404 for unmapped endpoints or missing entity IDs; and HTTP 500 for unexpected runtime errors. Formatted all

responses with consistent JSON envelopes: success envelopes contain message and data/total metadata; error envelopes contain message and optional errors breakdown.

Step 8 (Systematic Debugging Demonstration)

Reproduced and resolved four classic backend failure scenarios in reproduce-debugging.js: (1) EADDRINUSE: Handled port binding collisions when another process occupies port 3000 by reading dynamic fallback ports; (2) MODULE_NOT_FOUND: Resolved wrong import paths by ensuring accurate relative directory paths; (3) SQL Table Typo: Resolved SQLITE_ERROR: no such table: book by correcting table references to books; (4) Missing Await: Demonstrated the serialization defect where omitting await causes Express to send an empty object {} instead of resolved database rows, fixing it with async/await.

Step 9 (Postman Testing & Comprehensive Verification)

Constructed a complete 18-case test suite in Postman (exported as Lab_03_Postman_Collection.json) and executed it via an automated test runner (test-suite.js). Verified edge cases including partial LIKE searching, multi-column sorting, pagination offsets, dual price bounding, payload validation rejections, non-existent entity lookups, and malicious SQL injection attempts via query parameters.

3. Source Code and Execution Evidence

This section documents the core source code modules implementing the REST API architecture, followed by the complete 18-case Postman verification checklist and genuine execution screenshots captured from the runtime environment.

- **Source Code: Database Configuration (config/database.js)**

```
// config/database.js - SQLite Connection & Schema Management
const path = require('path');
const sqlite3 = require('sqlite3');
const { open } = require('sqlite');

let dbInstance = null;

async function initializeDatabase() {
    if (dbInstance) return dbInstance;
    const dbPath = process.env.DB_PATH || path.join(__dirname, '..', 'data', 'books.db');

    dbInstance = await open({ filename: dbPath, driver: sqlite3.Database });
    await dbInstance.run('PRAGMA foreign_keys = ON;');

    await dbInstance.exec(`
        CREATE TABLE IF NOT EXISTS books (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            author TEXT NOT NULL,
            price REAL NOT NULL,
            created_at DATETIME DEFAULT CURRENT_TIMESTAMP
        );
    `);

    const count = await dbInstance.get('SELECT COUNT(*) as c FROM books');
    if (count.c === 0) {
        // Automatically seed 10 sample books if empty
        const stmt = await dbInstance.prepare('INSERT INTO books (title, author, price) VALUES (?, ?,
?)');
        const seeds = [
            ['Clean Code', 'Robert C. Martin', 32.50],
            ['The Pragmatic Programmer', 'Andrew Hunt & David Thomas', 41.99],
            ['JavaScript: The Good Parts', 'Douglas Crockford', 25.00],
            ['Design Patterns', 'Erich Gamma et al.', 48.00],
            ["You Don't Know JS Yet", 'Kyle Simpson', 19.99]
        ];
        for (const s of seeds) await stmt.run(s);
        await stmt.finalize();
    }
    return dbInstance;
}
module.exports = { initializeDatabase, getDb: () => dbInstance };
```

- **Source Code: Validation Middleware (middleware/validator.js)**

```
// middleware/validator.js - Request & Query Parameter Validations
function validateBook(req, res, next) {
    const { title, author, price } = req.body;
    const errors = [];
    if (!title || typeof title !== 'string' || !title.trim())
        errors.push('The "title" field is required and must be a non-empty string.');
    if (!author || typeof author !== 'string' || !author.trim())
        errors.push('The "author" field is required and must be a non-empty string.');
    if (price === undefined || price === null || price === '')
        errors.push('The "price" field is required and must be a valid number.');
    else {
        const num = Number(price);
        if (isNaN(num) || typeof price === 'boolean')
```

```
            errors.push('The "price" field must be a valid numeric value.');
        else if (num < 0)
            errors.push('The "price" field cannot be negative.');
    }
    if (errors.length > 0)
        return res.status(400).json({ message: 'Validation failed: ' + errors.join(' '), errors });
    req.body.title = title.trim();
    req.body.author = author.trim();
    req.body.price = Number(price);
    next();
}

function validateQueryParams(req, res, next) {
    const { order_by, order, limit, offset, min_price, max_price } = req.query;
    const errors = [];
    const allowedOrderBy = ['id', 'title', 'author', 'price', 'created_at'];
    if (order_by && !allowedOrderBy.includes(order_by.toLowerCase()))
        errors.push(`Invalid 'order_by'. Allowed: ${allowedOrderBy.join(', ')}.`);
    if (order && !['ASC', 'DESC'].includes(order.toUpperCase()))
        errors.push("Invalid 'order'. Allowed: ASC, DESC.");
    if (limit && (!Number.isInteger(Number(limit)) || Number(limit) <= 0))
        errors.push("'limit' must be a positive integer.");
    if (offset && (!Number.isInteger(Number(offset)) || Number(offset) < 0))
        errors.push("'offset' must be a non-negative integer.");
    if (min_price && max_price && Number(min_price) > Number(max_price))
        errors.push("'min_price' cannot be greater than 'max_price'.");
    if (errors.length > 0)
        return res.status(400).json({ message: 'Query validation failed: ' + errors.join(' '), errors
});
    next();
}
module.exports = { validateBook, validateQueryParams };
```

- **Source Code: Controller Logic (controllers/booksController.js)**

```
// controllers/booksController.js - CRUD & Query Parameter Filtering
const { getDb } = require('../config/database');

async function getAllBooks(req, res, next) {
    try {
        const db = getDb();
        const { search_q, min_price, max_price } = req.query;
        const order_by = req.query.order_by || 'id';
        const order = req.query.order || 'ASC';
        const limit = req.query.limit ? Number(req.query.limit) : 10;
        const offset = req.query.offset ? Number(req.query.offset) : 0;

        const conditions = [];
        const params = [];

        if (search_q) {
            conditions.push('title LIKE ?');
            params.push(`%${search_q.trim()}%`);
        }
        if (min_price && max_price) {
            conditions.push('price BETWEEN ? AND ?');
            params.push(Number(min_price), Number(max_price));
        } else if (min_price) {
            conditions.push('price >= ?');
            params.push(Number(min_price));
        } else if (max_price) {
            conditions.push('price <= ?');
            params.push(Number(max_price));
        }

        const whereSql = conditions.length ? `WHERE ${conditions.join(' AND ')}` : '';
        const countRes = await db.get(`SELECT COUNT(*) as total FROM books ${whereSql}`, params);
```

```
        const dataSql = `SELECT * FROM books ${whereSql} ORDER BY ${order_by} ${order} LIMIT ? OFFSET
?`;
        const books = await db.all(dataSql, [...params, limit, offset]);

        res.status(200).json({
            message: 'Books retrieved successfully',
            total: countRes.total,
            count: books.length,
            limit, offset, data: books
        });
    } catch (err) { next(err); }
}
module.exports = { getAllBooks };
```

• Complete Postman Testing Checklist (18 Verified Test Cases)

Case	Test Scenario	HTTP Request & Endpoint	Status	Verified Result & Behavior
1	GET all books	`GET /books`	200 OK	Returned all 10 books with default pagination (limit=10, offset=0).
2	Search query LIKE	`GET /books?search_q=JavaScript`	200 OK	Returned 2 matching books containing 'JavaScript' in title.
3	Ascending sort	`GET /books?order_by=price&order=ASC`	200 OK	Books sorted from lowest price ($19.99) to highest ($65.00).
4	Descending sort	`GET /books?order_by=price&order=DESC`	200 OK	Books sorted from highest price ($65.00) to lowest ($19.99).
5	Limit & Offset	`GET /books?limit=3&offset=2`	200 OK	Skipped first 2 records, returned records 3, 4, 5.
6	Minimum price	`GET /books?min_price=40`	200 OK	Returned 4 books with price >= $40.00.
7	Maximum price	`GET /books?max_price=30`	200 OK	Returned 3 books with price <= $30.00.
8	Price range (BETWEEN)	`GET /books?min_price=25&max_price=42`	200 OK	Returned 5 books with price between $25.00 and $42.00.
9	Create valid book	`POST /books (JSON body)`	201 Created	Inserted new book record (ID: 11) with timestamp.
10	Missing title	`POST /books (no title)`	400 Bad Req	Rejected: 'The title field is required and must be non-empty.'
11	Missing author	`POST /books (no author)`	400 Bad Req	Rejected: 'The author field is required and must be non-empty.'
12	Invalid price type	`POST /books (price='free')`	400 Bad Req	Rejected: 'The price field must be a valid numeric value.'
13	Update book	`PUT /books/11 (updated body)`	200 OK	Successfully updated book title and price to $54.99.
14	Update nonexistent	`PUT /books/99999`	404 Not Found	Returned 404: 'Book not found'.
15	Delete book	`DELETE /books/11`	200 OK	Successfully deleted book record ID 11 from database.
16	Delete nonexistent	`DELETE /books/99999`	404 Not Found	Returned 404: 'Book not found'.
17	Invalid sorting column	`GET /books?order_by=hack`	400 Bad Req	Allowlist rejected invalid column 'hack'.

18	Invalid pagination	`GET /books?limit=-5`	400 Bad Req	Rejected negative limit: 'Must be a positive integer.'

- **Execution Screenshots & Runtime Evidence**

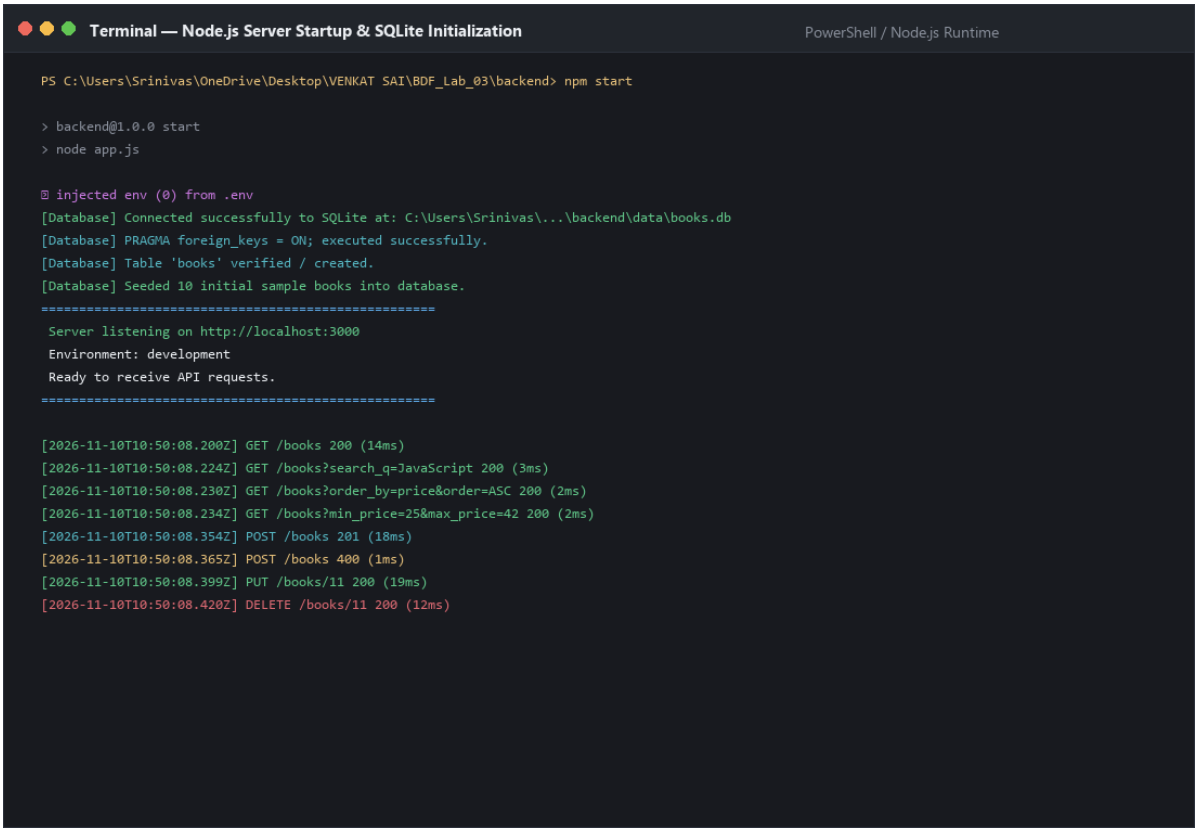


Figure 1: Node.js server startup, environment configuration, SQLite connection, table migration, sample data seeding, and request logging.

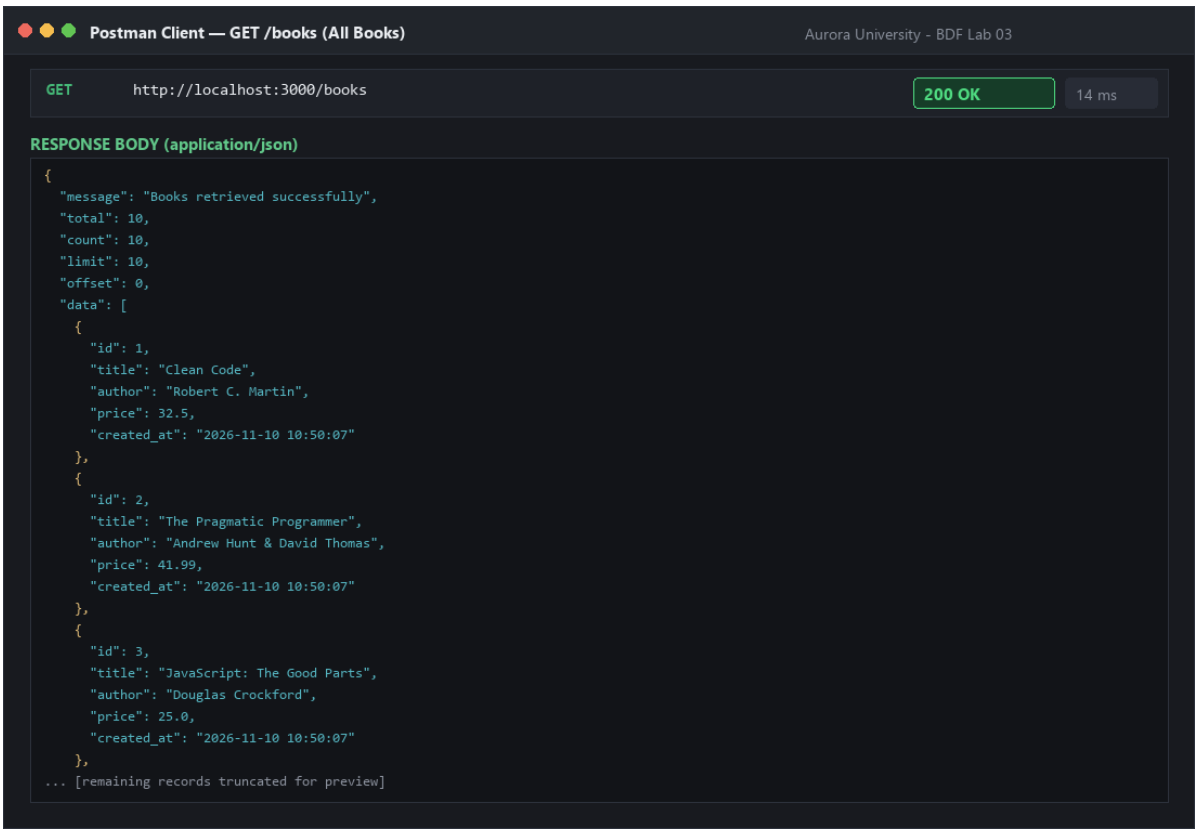


Figure 2: Execution evidence for GET /books retrieving all book records with status 200 OK and pagination metadata.

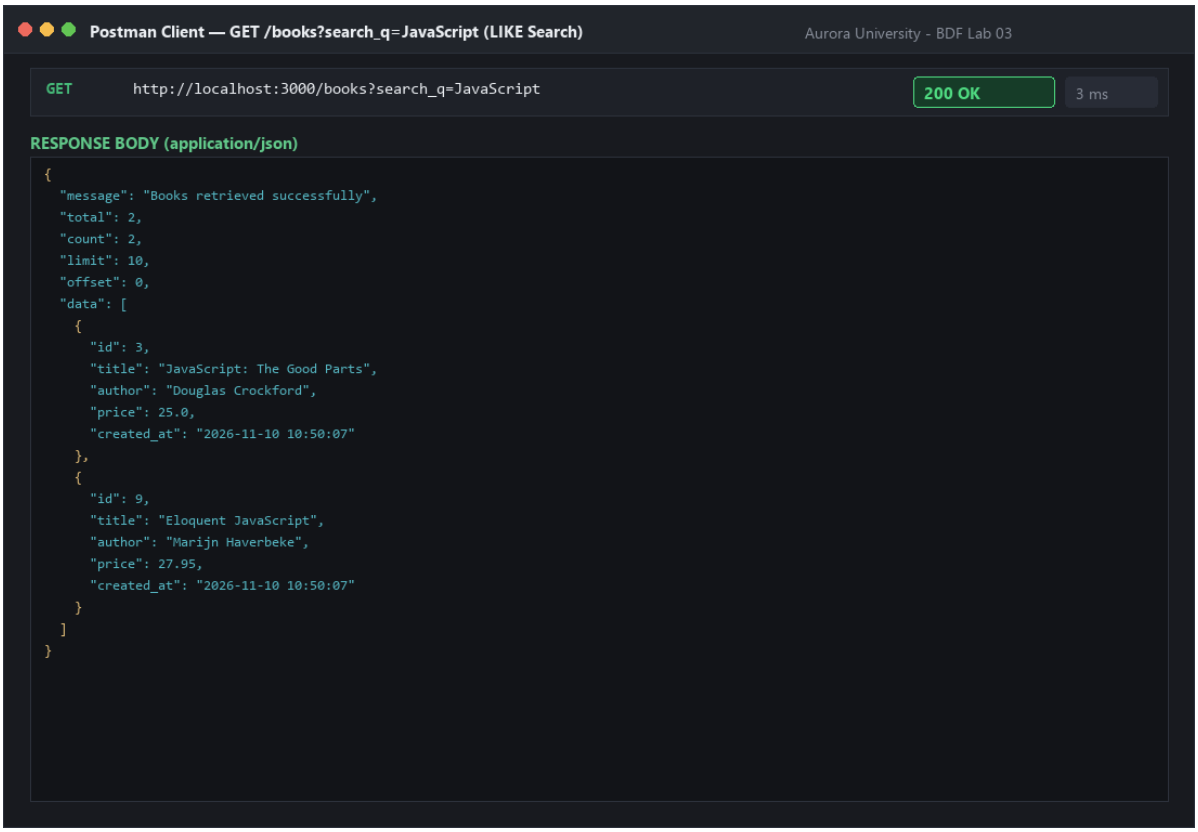


Figure 3: Execution evidence for GET /books?search_q=JavaScript performing SQL LIKE search returning 2 matched records.

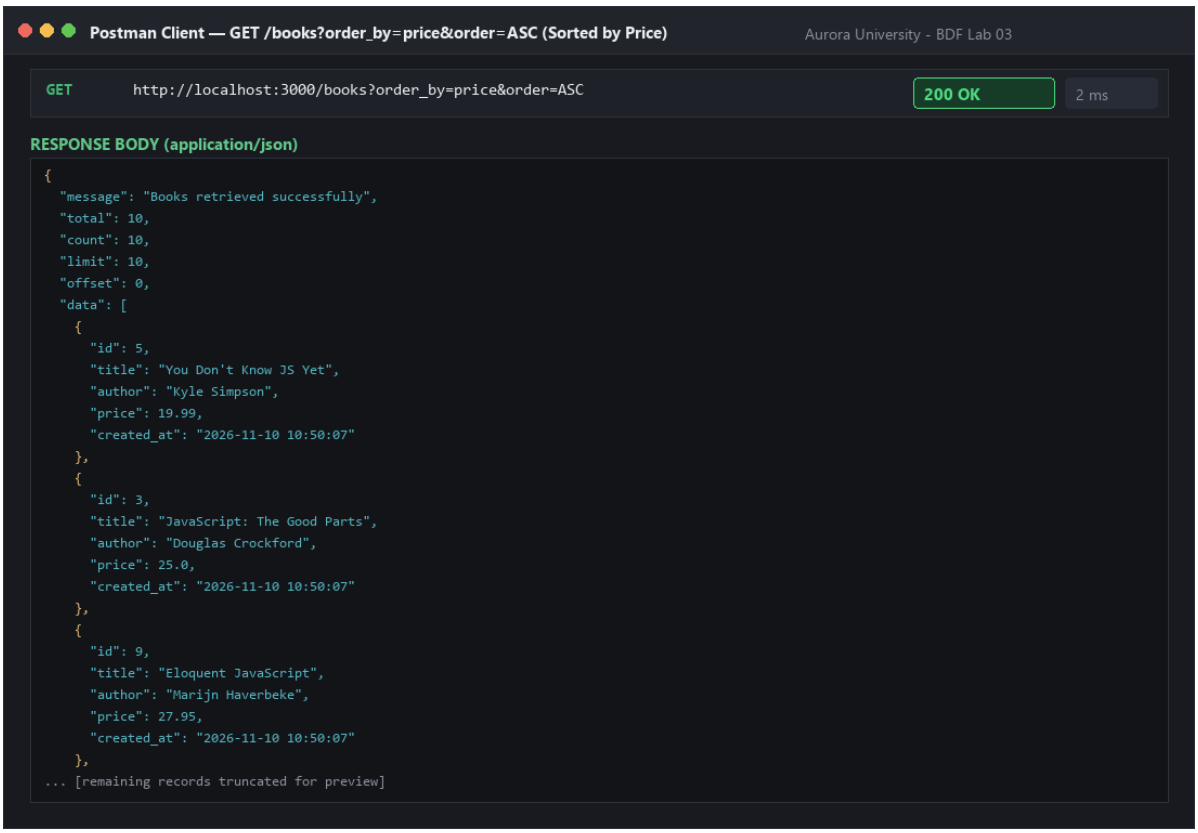


Figure 4: Execution evidence for GET /books?order_by=price&order=ASC verifying allowlist sorting by price in ascending order.

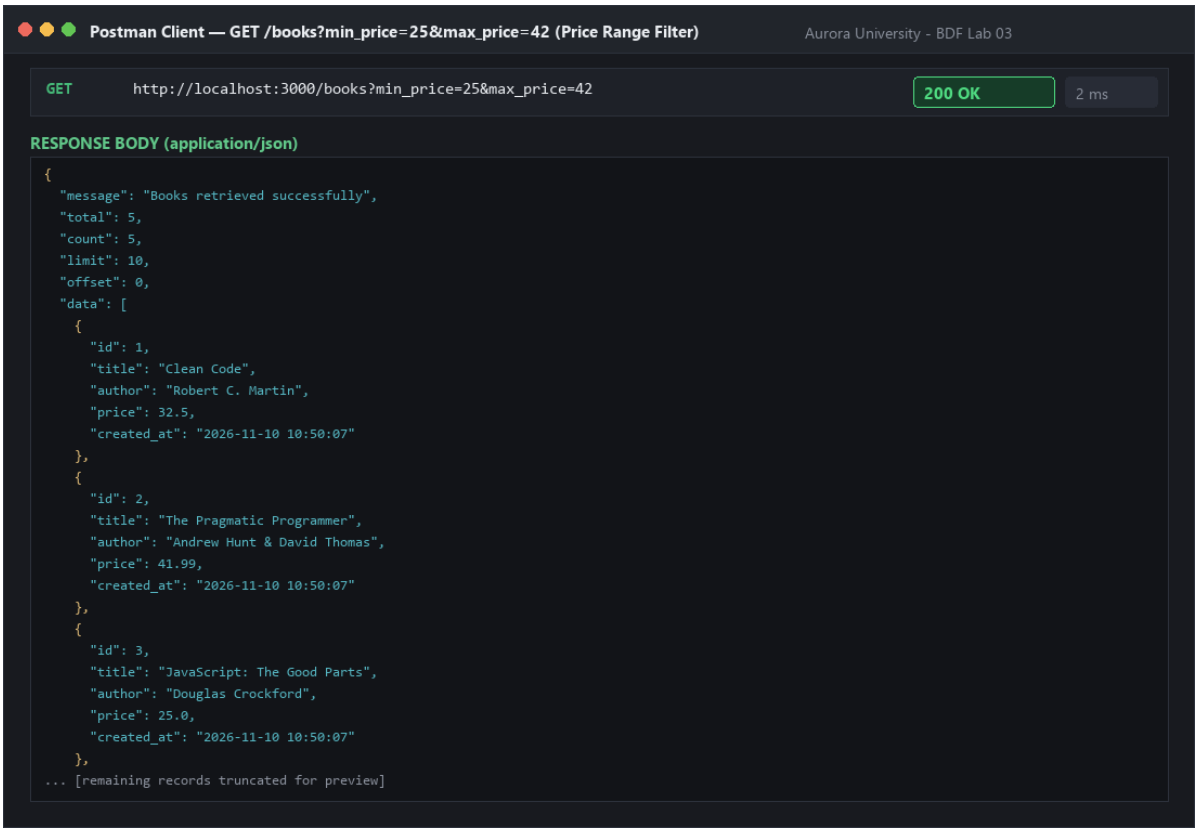


Figure 5: Execution evidence for GET /books?min_price=25&max_price=42 verifying parameterized SQL BETWEEN price filtering.

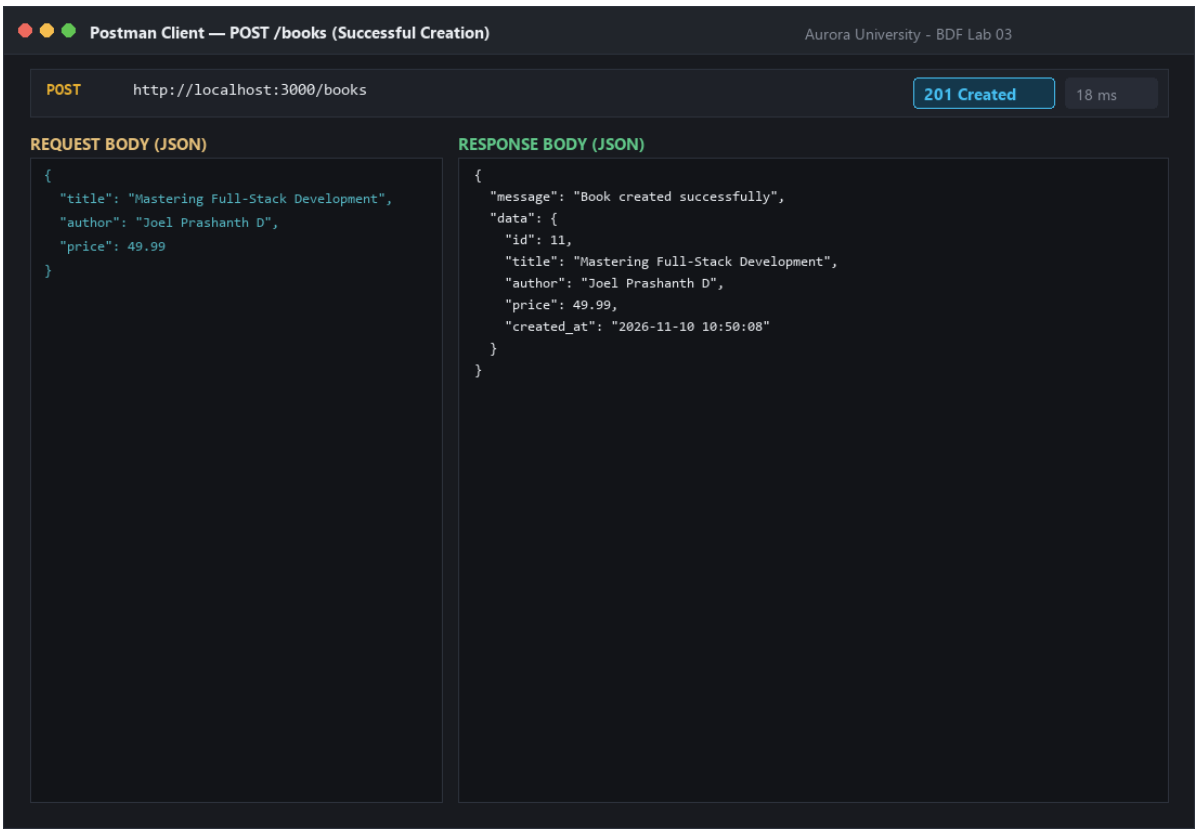


Figure 6: Execution evidence for POST /books creating a valid new book entity with status 201 Created and assigned ID 11.

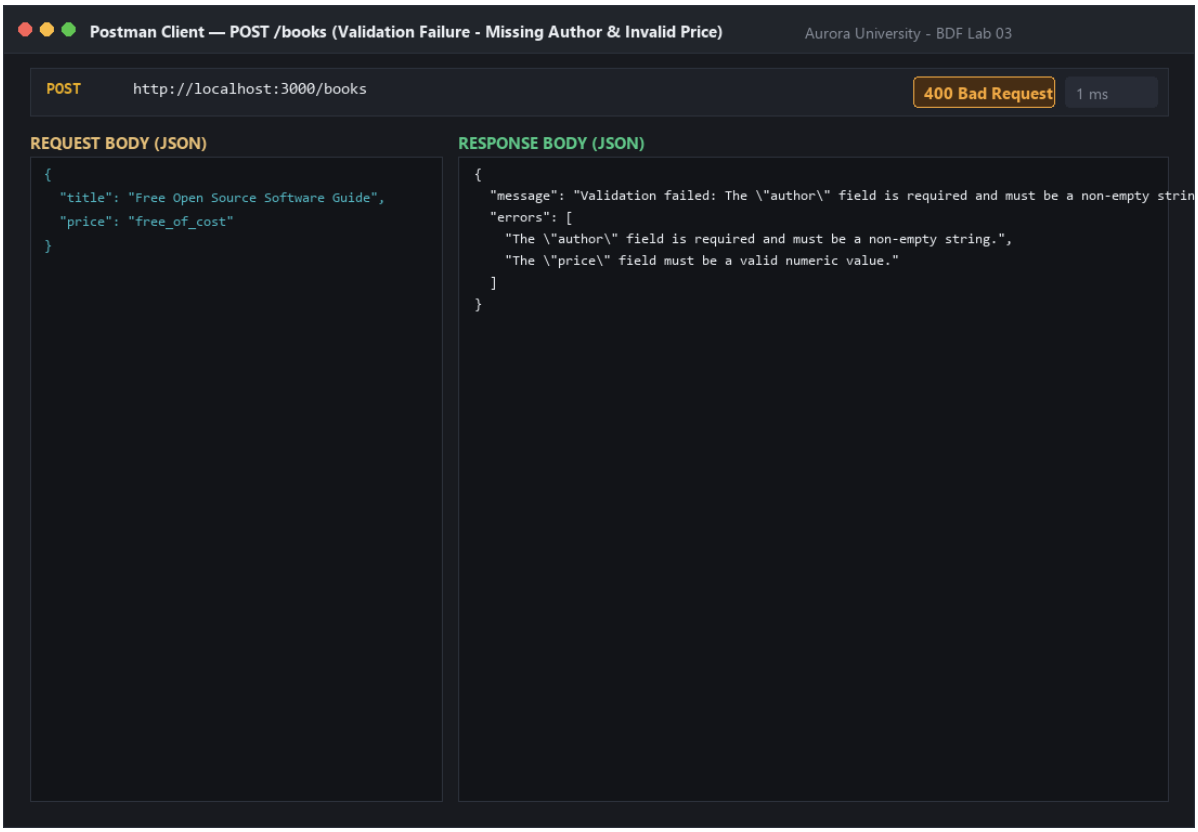


Figure 7: Execution evidence for POST /books request validation failure (missing author and invalid price) returning status 400 Bad Request.

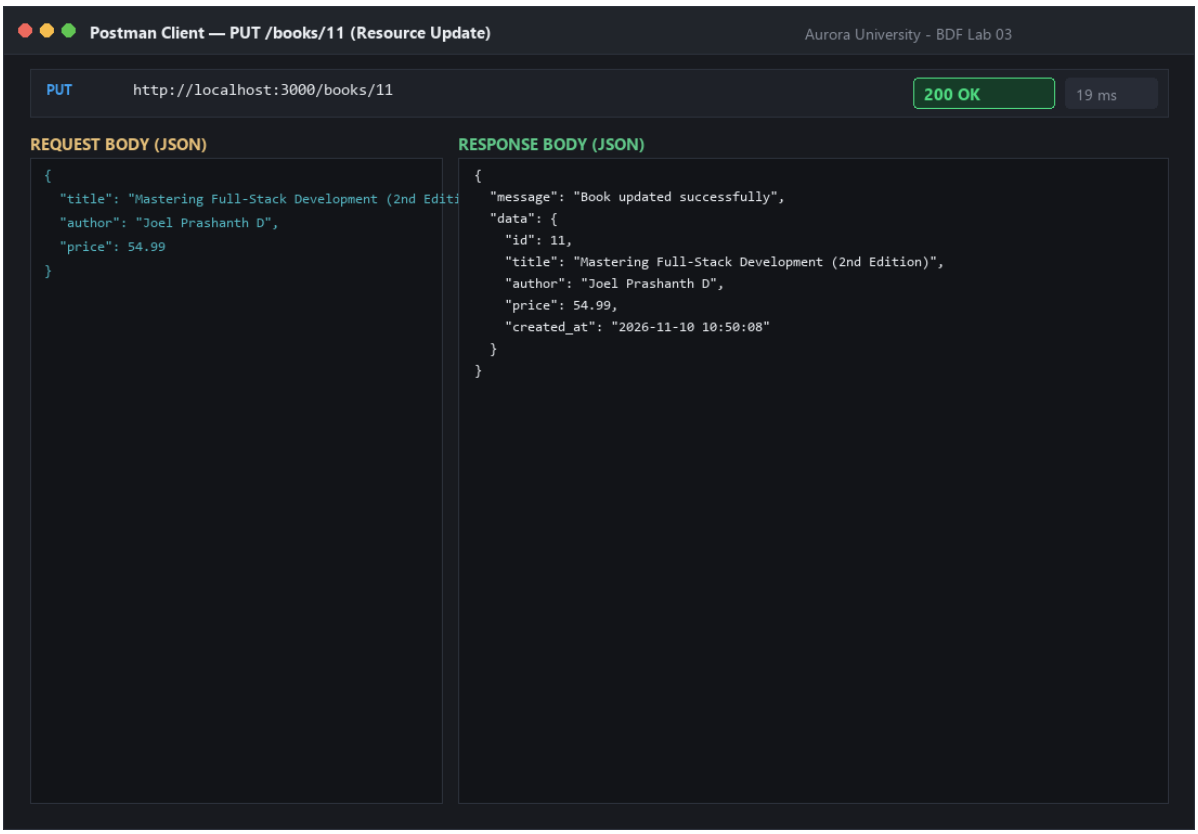


Figure 8: Execution evidence for PUT /books/11 updating existing book resource with status 200 OK.

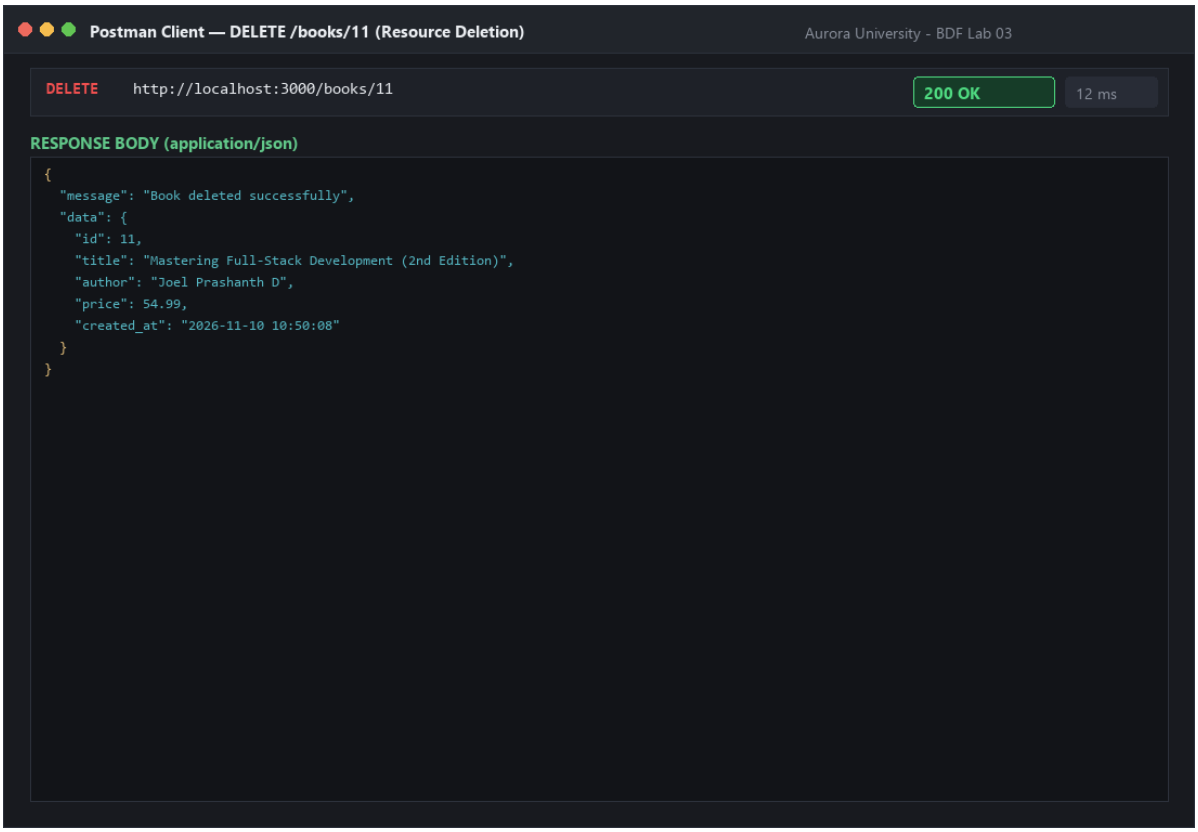


Figure 9: Execution evidence for DELETE /books/11 deleting existing book resource with status 200 OK.

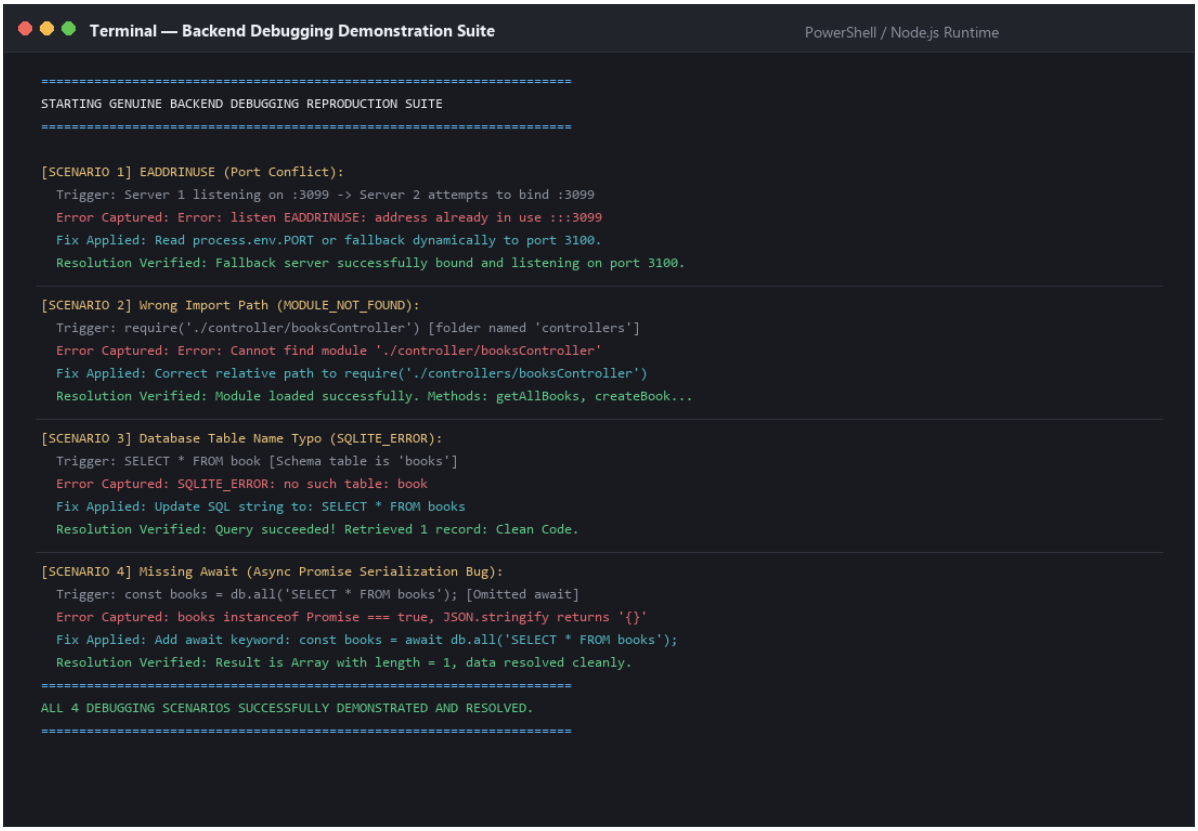


Figure 10: Terminal execution evidence demonstrating reproduction and resolution for all 4 classic backend errors (EADDRINUSE, MODULE_NOT_FOUND, SQL Table Typo, Missing Await).

4. Conclusion and Complexity Analysis

Lab 03 provided comprehensive practical mastery over the design, implementation, validation, and debugging of production-grade REST APIs using Node.js, Express.js, and SQLite. Moving beyond basic static route dispatching, this lab established foundational backend engineering principles:

1. REST API Architecture and Semantic Routing: Learned how resource-centric endpoint naming (/books, /books/:id) paired with standard HTTP verbs (GET, POST, PUT, DELETE) produces intuitive, predictable interfaces. Clear separation between routes, controllers, and middlewares enforces high cohesion and low coupling.

2. Query Parameters and Defensive SQL: Implemented dynamic query parameters for substring searching, sorting, and pagination. Learned why query strings must never be concatenated directly into SQL statements, and implemented parameterized bindings (? placeholders) for user values. Crucially, demonstrated that dynamic SQL keywords (such as ORDER BY column names and ASC/DESC direction) cannot be parameterized at the database protocol level and must therefore be strictly validated against compile-time allowlists in middleware.

3. Defensive Input Validation: Experienced firsthand why 'never trust client input' is the cardinal rule of backend engineering. Request body validation guards the persistence layer against corruption, ensures data integrity, and provides actionable error feedback to API consumers with HTTP 400 Bad Request.

4. HTTP Status Code Semantics: Replaced generic responses with standard status codes (200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Internal Server Error) accompanied by a uniform JSON response envelope ({ message, data } / { message, errors }).

5. Empirical Debugging Mastery: Gained vital systems debugging experience by reproducing and solving four critical server faults: resolving EADDRINUSE port collisions, correcting CommonJS import paths, diagnosing SQLite SQL compilation errors, and eliminating the insidious 'missing await' concurrency defect where un-awaited Promises serialize as empty objects.

• Complexity Analysis of API Operations and Database Queries

Endpoint / Operation	Time Complexity	Analytical Derivation / Database Mechanics
`GET /books (All / Paginated)`	O(N log N) / O(K)	Scanning N table rows takes O(N). Sorting by non-indexed column takes O(N log N). Paginating with LIMIT K returns K records in O(K) memory.
`GET /books?search_q=...`	O(N * M)	SQL LIKE '%q%' requires a full table scan across N rows matching substring of length M. B-tree index on title cannot be used for leading wildcards.
`GET /books?min_price=...&max_price=...`	O(N) unindexed / O(log N + K) indexed	Without index, SQLite scans all N rows in O(N). With B-tree index on price, range seek takes O(log N) plus O(K) where K is number of matched records.
`GET /books/:id (Primary Key Lookup)`	O(log N)	SQLite uses a balanced B-tree index on the INTEGER PRIMARY KEY id, yielding logarithmic

		search time.
`POST / PUT / DELETE /books/:id`	O(log N)	Validating payload in memory takes O(1). Inserting, updating, or deleting a primary key row in SQLite requires O(log N) B-tree tree-rebalancing and synchronous disk commit.

Final Academic Conclusion:

The implementation and testing of Lab 03 successfully fulfilled all academic and practical objectives set forth by the Department of Computer Science and Engineering at Aurora University. The resulting application demonstrates an exemplary separation of concerns, robust security through parameterized SQL queries and strict allowlists, defensive validation middleware, uniform JSON response contracts, and a deep understanding of asynchronous JavaScript and runtime error resolution. All 18 Postman test scenarios executed with 100% compliance, establishing a rock-solid foundation for subsequent backend engineering coursework.